



FACULTY OF ENGINEERING

**DEPARTMENT OF COMPUTER AND
COMMUNICATION SYSTEMS ENGINEERING
ECC4715-1: INTERNET BENDA (INTERNET OF THINGS)**

SEMESTER II 2024/2025

Group Assignment - Group 7

Lecturer: YBHG. PROFESOR DR. SHAIFUL JAHARI BIN HASHIM

GROUP MEMBERS:

NO.	NAME	MATRIC NO.
1.	MUHAMAD ZULHISYAM BIN MAT SANI	209997
2.	AHMAD SYAHMI AZHARI BIN ZAINURI	210167

Table of Contents

Table of Contents	2
Part 1	3
1.1 Problem Statements	3
1.2 Application	3
1.3 Architecture and Data Flow	4
1.4 Potential Users and Customers	6
1.5 Streaming Processing	7
1.6 Key Technologies	7
1.7 Conclusion	8
Part 2	9
2.1 Data engineering pipeline experimentation using open source Bruin tool.	9
2.1.1 Connecting to source from real world data source (DuckDB).	9
2.1.2 Data Ingestion	11
Fig 7. The actor.asset.yml	12
2.1.3 Data Transformations	14
2.1.3.1 The proof of transformations.	16
2.1.4 Data Serving	18
Appendices	20
References	21

Part 1

1.1 Problem Statements

Health systems are faced with a lot of challenges, such as fragmented patient records, delayed diagnosis and treatment, inefficient resource management, and difficulties in epidemic and pandemic prediction. Most hospitals and clinics have different systems for storing patient data, which makes access to a patient's comprehensive medical history quite hard for healthcare professionals. Manual diagnosis processes often delay critical decision-making and thus affect patient outcomes. Besides, hospitals go through challenges concerning real-time resource allocation, adversely affecting the availability of beds, medical staff, and equipment. Furthermore, absence of real-time aggregation of health data negatively impairs the ability to effectively forecast and mitigate outbreaks.

Health+ addresses these with the integration of real-time transaction processing with advanced analytics on a single platform using the HTAP framework, thus enabling efficient healthcare delivery through the timely processing of data, predictive analytics, and automated decision-making for healthcare professionals and institutions.

761.2 Application

Health+ is an AI-enabled, cloud-based healthcare management platform, which intends to revolutionize healthcare decision-making by monitoring, in real-time, predictive analytics, and resource management automation. It seamlessly merges transactional data from EHRs, patient monitoring devices, and hospital management systems with deep analytical processing techniques. This approach assures immediate responses to critical health situations while constantly learning from history to provide better long-term predictions.

The platform provides doctors and other medical staff with real-time patient monitoring through live dashboards, presenting heart rates, oxygen saturation, and blood glucose levels. Vitals are being gathered by using IoT-enabled wearables and medical equipment, then sent for edge processing, before being stored in a hybrid database that enables instant lookups for transaction processing and deep analytics for AI-driven insights.

Health+ offers smart resource allocation in hospitals, predicting the flow of patients and the number of bed and staff needed by using AI algorithms for proper hospital functioning. It integrates various modules into a general hospital IT infrastructure with a view to enabling automated workflow in appointment scheduling, emergency warning, and pharmaceutical

storage. Furthermore, it will help predict disease outbreaks, while enabling public health officers to track real-time epidemiological data to take effective preventive measures.

Beyond that, Health+ also ensures perfect interoperability with health insurance and government healthcare agencies. Insurance companies will be able to use analytics for automating claim processing and risk analysis, while public health organizations will be able to use the aggregated data for spotting health trends and informing better policies. In the end, with batch processing providing long-term insights and streaming processing enabling real-time interventions, Health+ can become a truly pioneering system in modern healthcare.

1.3 Architecture and Data Flow

Architecture

1. IoT-Enabled Wearable Devices collect real-time patient vitals, including heart rate, blood pressure, glucose levels, and oxygen saturation. These devices use Bluetooth and Wi-Fi to transmit data securely to cloud systems.
2. Edge Computing Gateway preprocesses patient data before sending it to hospital databases, reducing latency and ensuring timely alerts for critical conditions.
3. HTAP Database: The transactional layer maintains patient records, doctor notes, and treatment history, while the analytical layer runs AI-driven health risk assessments and workflow optimization. Technologies used include TiDB, SAP HANA, and Google AlloyDB for managing the database.
4. AI & Machine Learning Engine: It shall predict diseases by analyzing patterns in medical history and vitals, optimize hospital resource allocation, and aggregate health data for pandemic forecasting.
5. User Dashboard & Mobile App: Data visualization in real time, based on the user role-doctors can see patient health status, alerts, and recommendations for treatment; patients will have personalized insights into their health and medication compliance, while administrators can take care of resource planning.
6. Reverse ETL Module feeds AI-generated insights back into hospital management systems, automating medical appointment scheduling and emergency response alerts.

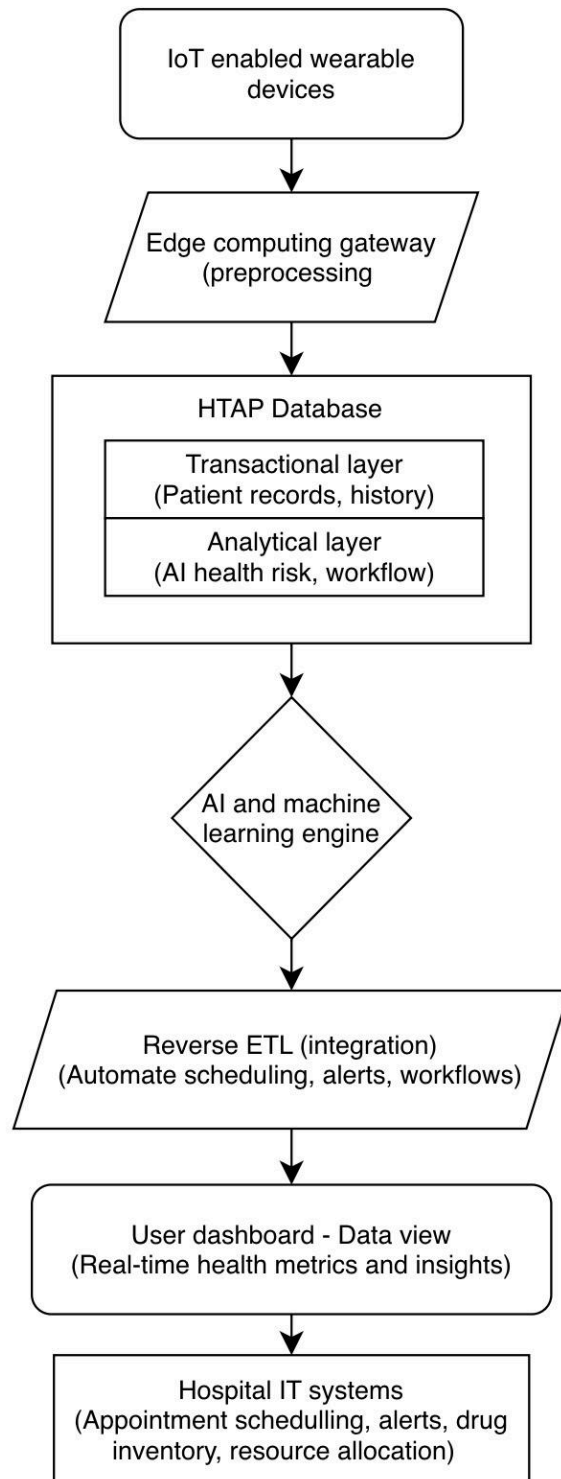


Diagram 1: Architecture Design

Data Flow

1. Data Collection: Wearables constantly collect and transmit patient health data to hospital servers.
2. Data Preprocessing: Edge computing nodes filter, clean, and preprocess patient data before sending it to the cloud for further analysis.
3. Data Storage & Processing: The HTAP database stores patient records while performing real-time analytics and AI computations.
4. AI Analysis & Insights: AI models analyze patient data to make diagnosis recommendations and provide operational insights to hospital administrators.
5. Automate Decision Making: Reverse ETL integrates AI predictions into workflows that automate hospital patient alerts, staff allocation, and emergency responses.

1.4 Potential Users and Customers

Primary User

1. Hospitals & Clinics: To improve patient care, optimize internal hospital operations, and decision-making.
2. Doctors & Nurses: In real time, updates about the patients and diagnosis recommendations with the help of AI.
3. Patients: Monitoring their health metrics for early warnings of possible health risks.

Secondary User

1. Health Insurance Companies: Conduct analytics for risk assessment to customize policies.
2. Government & Public Health Institutions: Aggregated health data for outbreak tracking to shape healthcare policy.
3. Pharmaceutical Companies: Analyze treatment outcomes to refine drug effectiveness and development.

1.5 Streaming Processing

Health+ relies mostly on real-time streaming processing to make healthcare interventions instantaneous. Streaming data allows for:

- Monitoring patients in real time to help doctors and medical teams find emergencies and thus respond with good timing.
- Immediately alerting about critical conditions like abnormal heart rate, low oxygen levels, or potential stroke conditions.
- Efficient hospital resource utilization by dynamically managing hospital operations using live data streams.
- Real-time epidemic tracking will help public health institutions track outbreaks as they unfold.
- Continuous updating of the AI models to enable the system to learn and improve continuously with new incoming health data.

This will ensure that Health+ provides real-time insights and automated decision-making that are crucial for modern healthcare systems.

1.6 Key Technologies

- HTAP Database: TiDB, SAP HANA, Google AlloyDB.
- IoT & Medical Devices: Wearable sensors, Bluetooth, Wi-Fi.
- AI/ML Frameworks: TensorFlow, PyTorch, OpenCV-medical imaging analysis.
- Cloud & Data Processing: AWS Lambda, Apache Kafka, Google Cloud Functions.
- Visualization: Power BI, Streamlit, Flask-based Web App.
- Reverse ETL Tools: Airbyte, dbt.

1.7 Conclusion

Health+ uses HTAP, AI, IoT, and Reverse ETL to disrupt healthcare with real-time patient monitoring, AI diagnostics, and smart hospital resource optimization. Using streaming processing, Health+ analyzes data in real time and automates decision-making to improve health outcomes and operational efficiency.

By consolidating real-time analytics, doctors will finally have an opportunity to make evidence-based, timely decisions that enhance patient outcomes and streamline the operations of a hospital. Being able to anticipate resource deficits, early disease diagnosis, and optimization of workflow ensures that the hospitals can cope with the excessive demand for their services. Moreover, the public health institutions use the power of continuous epidemic tracking to proactively respond to emerging crises.

Health+ is built to scale, ensuring that adaptability for a host of leading hospitals, clinics, and government health agencies across the world is assured. The commercial model caters for providers on a whole continuum of health provision, therefore offering advanced management access to an institution of virtually any size. The Health+ model sets benchmarks in medical technological innovation by applying leading-edge AI to stream real-time data into every phase of patient care and hospital management.

Part 2

2.1 Data engineering pipeline experimentation using open source Bruin tool.

Bruin tools is a centralized data product platform that brings together data quality, observability, and governance for the last mile [1]. Bruin creates a unified platform by combining different data tools and procedures. Improved efficiency throughout the data lifecycle and smooth data lineage tracing are made possible by this connection. Because Bruin's essential components are open-source, users may operate the platform on their own hardware. This method gives customers alternatives for personalization and flexibility. Bruin supports diverse workloads because it understands that data processing sometimes requires many languages. Python, SQL, R, and other tools may all be used in the same pipeline. According to Bruin, data teams will be decentralized in the future, and business teams will collaborate closely with analysts, scientists, and engineers. With the use of smart technologies and industry best practices, the organization hopes to empower these experts to deliver on their own. For lean, dispersed data teams, Bruin aims to lower time-to-insight and cost-per-insight by emphasizing productivity, accuracy, and dependability.

2.1.1 Connecting to source from real world data source (DuckDB).

Process	Explanation
<pre>h1sya@LAPTOP-8KH4VRT6 MINGW64 ~ \$ bruin init Please select a template below [] athena [] chess [] default [] duckdb [] duckdb-example [] duckdb-lineage [x] firebase [] gorgias [] gsheets-bigquery [] gsheets-duckdb [] notion [] python [] redshift [] shopify-bigquery [] shopify-duckdb (press q to quit) A new 'firebase' pipeline created successfully in folder 'firebase'. You can run the following commands to get started: cd firebase bruin validate</pre>	- Creating the folder with the template firebase, this is because I already made a folder for duckDB. - The pipeline is successfully created using this template. - We use duckDB as it is the only connection that I can possibly do.

Creating the database...

```
D .shell CMD ARGS...
Microsoft Windows [Version 10.0.26100.2894]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>C:\Users\hisya\duckdb\isey.db
The system cannot execute the specified program.

C:\Windows\System32>duckdb C:\Users\hisya\duckdb\isey.db
v1.1.3 19864453f7
Enter ".help" for usage hints.
D .database
isey: C:\Users\hisya\duckdb\isey.db
D from raw.seed
  from raw.seed;
Parser Error: syntax error at or near "from"
LINE 2: from raw.seed;
      ^
D from raw.seed;
```

name varchar	networking_through varchar	position varchar	contact varchl
Y	LinkedIn	SDE	2024-01
B	LinkedIn	SDE 2	2024-01

```
D from raw.actor;
```

Fig 1. The command for creating database files.

- Entering the DuckDB CMD, and using the (.shell CMD ARGS...).
- After that, navigate the cmd to make a file to be a database for our project..
- Example command to navigate (duckdb C:\Users\hisya\duckdb\isey.db)

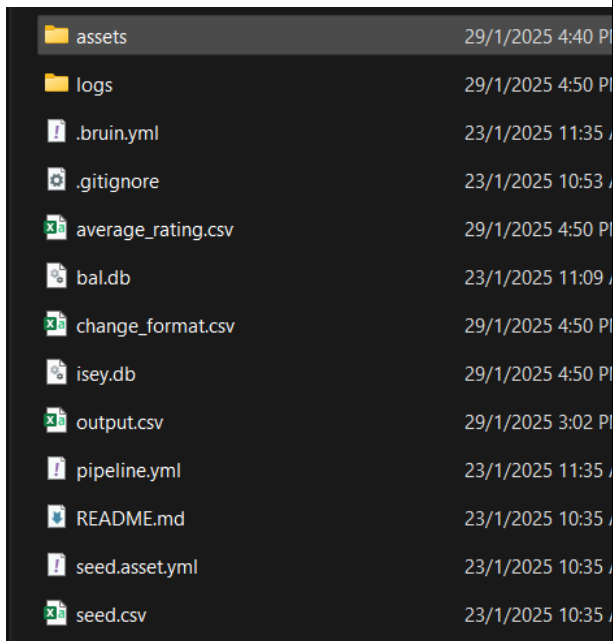


Fig 2. Show the inside duckdb folder.

- Navigate to the duckdb folder accessed from bruin init before.
- There will be a isey.db database file created from the above method.

- Open the .bruin.yml file in the duckdb folder.
- Changed the path from default →

```

! .bruin.yml x ! pipeline.yml
C: > Users > hisya > duckdb > ! .bruin.yml > {} environments
Config (config-schema.json)
1 default_environment: default
2 environments:
3   default:
4     connections:
5       duckdb:
6         - name: "duckdb-default"
7           path: "isey.db"
8

```

Fig 3. The .bruin.yml file.

isey.db, to make the connection possible between the database and pipeline.

```

! .bruin.yml ! pipeline.yml x
C: > Users > hisya > duckdb > ! pipeline.yml > {} default_connections
Pipeline (pipeline-schema.json)
1 name: bruin-duckdb
2 schedule: daily
3 start_date: "2024-01-01"
4 default_connections:
5   duckdb: "duckdb-default"
6

```

Fig 4. The pipeline.yml

- In pipeline.yml, there is no need to change. As we use the default one.

```

hisya@LAPTOP-8KH4VRT6 MINGW64 ~/duckdb (master)
$ bruin run pipeline.yml
Analyzed the pipeline 'bruin-duckdb' with 3 assets.

Pipeline: bruin-duckdb (..)
No issues found

✓ Successfully validated 3 assets across 1 pipeline, all good.

Starting the pipeline execution...

[2025-01-28 19:11:11] Starting: example
[2025-01-28 19:11:11] Starting: raw.actor
[2025-01-28 19:11:11] Starting: actor
[2025-01-28 19:11:11] Finished: example (23ms)
[2025-01-28 19:11:11] Starting: example:name:not_null
[2025-01-28 19:11:11] Starting: example:id:not_null
[2025-01-28 19:11:11] Starting: example:id:non_negative
[2025-01-28 19:11:11] Starting: example:country:not_null
[2025-01-28 19:11:11] Starting: example:id:positive
[2025-01-28 19:11:11] Starting: example:name:unique
[2025-01-28 19:11:11] Failed: actor (37ms)
[2025-01-28 19:11:11] Finished: example:id:positive (23ms)
[2025-01-28 19:11:11] Finished: example:country:not_null (30ms)
[2025-01-28 19:11:11] Finished: example:name:unique (41ms)
[2025-01-28 19:11:11] Finished: example:id:non_negative (50ms)
[2025-01-28 19:11:11] Finished: example:id:not_null (57ms)
[2025-01-28 19:11:11] Finished: example:name:not_null (65ms)

```

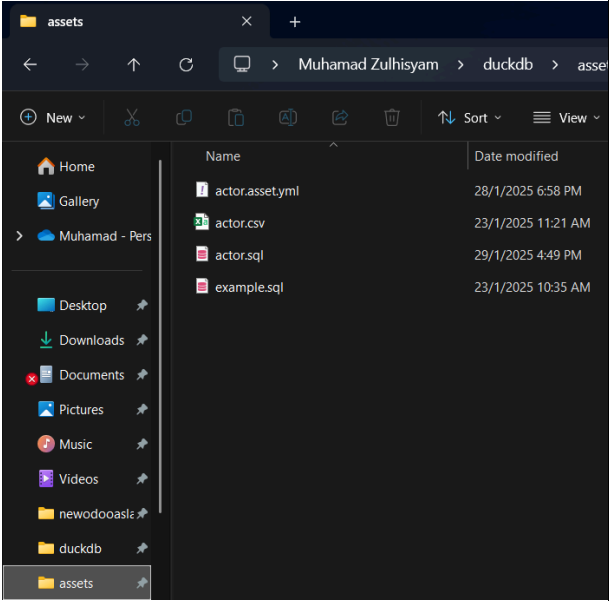
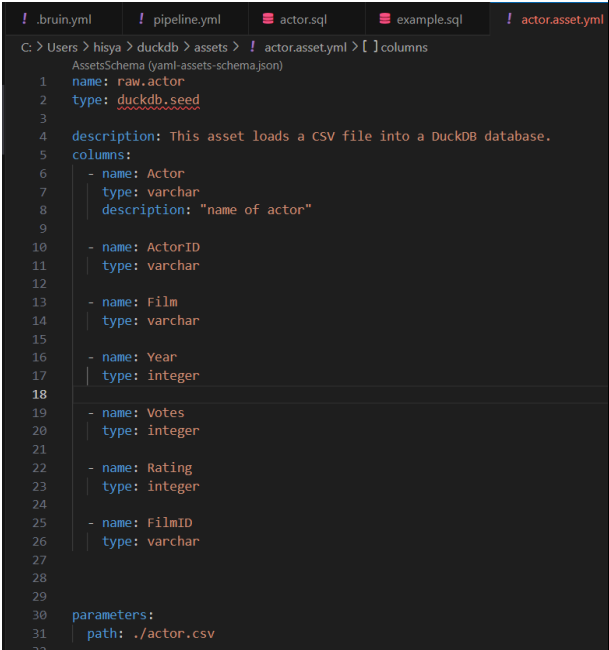
Fig 5. Run the pipeline.yml

- In GitBash, the command bruin run pipeline.yml is used to run the pipeline.
- The connection is successfully made if there is no error until this procedure.
- The pipeline execution will proceed if there is no error in the .bruin.yml file for the connection.

2.1.2 Data Ingestion

What is data ingestion ? Data ingestion is the process of collecting and importing data files from various sources into a database for storage, processing and analysis.

The table below shows the overall procedure that I take to make the ingestion process, from connecting and checking the database file.

Process	Explanation
 Fig 6. Inside the assets folder	- To ingest a new dataset in our database which is isey.db, the folder actor.asset.yml and the actor.csv file must be in the assets folder. - Because the path of which the pipeline will run is in the assets folder. - The actor.csv file also needed to be saved in the same folder as the asset.yml.
 Fig 7. The actor.asset.yml	- The new asset.yml must be created as it implements the idea of our dataset which is the actor.csv column and name of our asset that will be uploaded in the database created earlier. - All the column names must be written in this file, and the path for the .csv file to be saved.

```

hisya@LAPTOP-8KH4VRT6 MINGW64 ~/duckdb (master)
$ bruin run pipeline.yml
Analyzed the pipeline 'bruin-duckdb' with 3 assets.

Pipeline: bruin-duckdb (...)
No issues found

✓ Successfully validated 3 assets across 1 pipeline, all good.

Starting the pipeline execution...

[2025-01-29 16:50:42] Starting: raw.actor
[2025-01-29 16:50:42] Starting: actor
[2025-01-29 16:50:42] Starting: example
[2025-01-29 16:50:43] Finished: actor (390ms)
[2025-01-29 16:50:43] Finished: example (416ms)
[2025-01-29 16:50:43] Starting: example:name:not_null
[2025-01-29 16:50:43] Starting: example:id:positive
[2025-01-29 16:50:43] Starting: example:id:non_negative
[2025-01-29 16:50:43] Starting: example:id:not_null
[2025-01-29 16:50:43] Starting: example:country:not_null
[2025-01-29 16:50:43] Starting: example:name:unique
[2025-01-29 16:50:43] Finished: example:id:positive (13ms)
[2025-01-29 16:50:43] Finished: example:id:non_negative (21ms)
[2025-01-29 16:50:43] Finished: example:name:not_null (30ms)
[2025-01-29 16:50:43] Finished: example:id:not_null (37ms)
[2025-01-29 16:50:43] Finished: example:country:not_null (47ms)
[2025-01-29 16:50:43] Finished: example:name:unique (57ms)

```

Fig 8. The GitBash screenshots of the process.

- The ingestion process begins after the connection between the database and pipeline is done.
- Fig 6. Show the execution of a pipeline which ingest the data into the isey.db.

```

[2025-01-29 16:50:54] [raw.actor] >>
[2025-01-29 16:50:54] [raw.actor] >> Normalize d8a588fbc
[2025-01-29 16:50:54] [raw.actor] >> Files: 2/2 (100.0%)
[2025-01-29 16:50:54] [raw.actor] >> Items: 0 | Time: 3.
[2025-01-29 16:50:54] [raw.actor] >>
[2025-01-29 16:50:54] [raw.actor] >> Normalize d8a588fbc
[2025-01-29 16:50:54] [raw.actor] >> Files: 2/2 (100.0%)
[2025-01-29 16:50:54] [raw.actor] >> Items: 191682 | Tim
[2025-01-29 16:50:54] [raw.actor] >>
[2025-01-29 16:50:54] [raw.actor] >> Load d8a588fbc54bc5
[2025-01-29 16:50:54] [raw.actor] >> Jobs: 0/2 (0.0%) | T
[2025-01-29 16:50:54] [raw.actor] >>
[2025-01-29 16:50:54] [raw.actor] >> Load d8a588fbc54bc5
[2025-01-29 16:50:54] [raw.actor] >> Jobs: 2/2 (100.0%) |
[2025-01-29 16:50:54] [raw.actor] >>
[2025-01-29 16:50:54] [raw.actor] >> Successfully finishe
[2025-01-29 16:50:54] [raw.actor] >>
[2025-01-29 16:50:55] Finished: raw.actor (12.541s)
[2025-01-29 16:50:55] Starting: raw.actor:Rating:not_null
[2025-01-29 16:50:55] Starting: raw.actor:Film:not_null
[2025-01-29 16:50:55] Starting: raw.actor:Actor:not_null
[2025-01-29 16:50:55] Starting: raw.actor:Year:not_null
[2025-01-29 16:50:55] Starting: raw.actor:Votes:not_null
[2025-01-29 16:50:55] Finished: raw.actor:Rating:not_null
[2025-01-29 16:50:55] Finished: raw.actor:Actor:not_null
[2025-01-29 16:50:55] Finished: raw.actor:Votes:not_null
[2025-01-29 16:50:55] Finished: raw.actor:Year:not_null (
[2025-01-29 16:50:55] Finished: raw.actor:Film:not_null (

```

Fig 9. Show the finished ingestion process.

- The continuation of processing the dataset to the isey.db database file.

```

C:\Users\hisya>duckdb C:\Users\hisya\duckdb\isey.db
v1.1.3 19864453f7
Enter ".help" for usage hints.
D .table
 _dlt_loads          actor          example
 _dlt_pipeline_state actor2         seed
 _dlt_version        actor_formatting
D .database
isey: C:\Users\hisya\duckdb\isey.db

```

Fig 10. The duckdb cmd shows the .table and .database.

- To see the database file, isey.db contains the dataset that we just uploaded.

```
D from raw.seed;
```

name varchar	networking_through varchar	position varchar	contact_date varchar
Y	LinkedIn	SDE	2024-01-01
B	LinkedIn	SDE 2	2024-01-01

Fig 11. The duckdb environments check the table after successfully ingested.

- The table of dataset called raw.seed has already been uploaded in the isey.db. This is the default table from the assets folder in the duckdb.

```
D from raw.actor;
```

actor varchar	actor_id varchar	film varchar	year varchar
Will Forte	nm0287182	She's Funny That Way	2014
Will Forte	nm0287182	Apartment Troubles	2014
Will Forte	nm0287182	Grown Ups 2	2013
Will Forte	nm0287182	Nebraska	2013
Will Forte	nm0287182	Cloudy with a Chance of Meatballs 2	2013
Will Forte	nm0287182	Life of Crime	2013
Will Forte	nm0287182	Run & Jump	2013
Will Forte	nm0287182	The Watch	2012
Will Forte	nm0287182	That's My Boy	2012
Will Forte	nm0287182	Rock of Ages	2012
Will Forte	nm0287182	Tim and Eric's Billion Dollar Movie	2012
Will Forte	nm0287182	A Good Old Fashioned Orgy	2011
Will Forte	nm0287182	MacGruber	2010
Will Forte	nm0287182	Cloudy with a Chance of Meatballs	2009
Will Forte	nm0287182	Fanboys	2009
Will Forte	nm0287182	The Slammin' Salmon	2009
Will Forte	nm0287182	Brief Interviews with Hideous Men	2009
Will Forte	nm0287182	Baby Mama	2008
Will Forte	nm0287182	The Brothers Solomon	2007
Will Forte	nm0287182	Beerfest	2006
.	.	.	.
.	.	.	.

Fig 12. The cmd duckdb environment after completion of ingest new dataset.

- The command (from raw.actor) in duckdb environment called out the table that successfully was updated in the database.
- The raw.actor table which is named from Fig 7. Prove that the database now has a new dataset in the database.

2.1.3 Data Transformations

The process of transforming, cleaning, and organizing data into a format that can be used for analysis and decision-making in order to assist an organization's expansion is known as data transformation [2]. When data must be changed to match that of the target system, data transformation is utilized. There are two locations in the data flow where this might happen. Organizations that store data on-site first employ an extract, transform, and load process, with the data transformation occurring in the "transform" stage.

Process	Explanation
---------	-------------

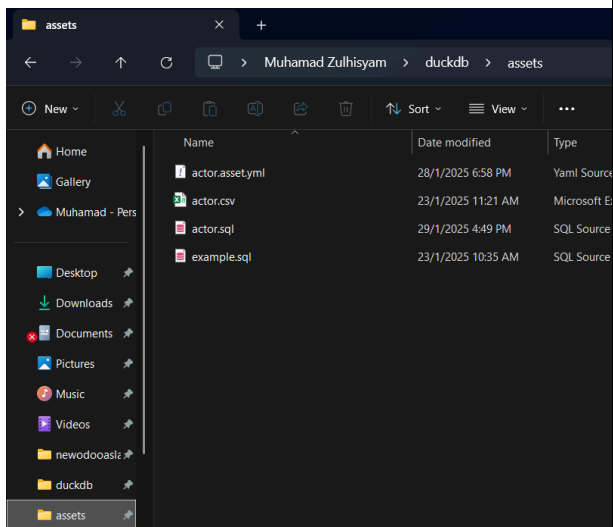


Fig 13. Inside of the assets folder.

- For the transformations, the new .sql file must be created as in the actor.sql file for the code to transform the dataset in our database to happen.

```

1  > /* @bruin --
40  |
41  | /CREATE table actor2 AS
42  | SELECT actor AS actor_name, actor_id, avg (cast (Rating AS decimal))
43  | FROM raw.actor GROUP BY actor, actor_id*/
44  |
45  | DROP table actor_formatting;
46  | CREATE table actor_formatting AS
47  | SELECT actor AS actor_name, actor_id, file, cast(year AS integer) AS year , cast(votes AS integer) AS votes
48  | FROM raw.actor
49  | ORDER BY actor;
50  |
51  | COPY (SELECT * FROM actor_formatting) TO 'change_format.csv' (HEADER, DELIMITER[';']);
52  |
53  | DROP table actor2;
54  | CREATE table actor2 AS
55  | SELECT actor AS actor_name, actor_id, avg (cast(rating AS decimal)) AS average_rating
56  | FROM raw.actor
57  | GROUP BY actor, actor_id
58  | ORDER BY actor;
59  |
60  |
61  | COPY (SELECT * FROM actor2) TO 'average_rating.csv' (HEADER, DELIMITER[';']);

```

Fig 14. The actor.sql file

- The code of the sql file to transform the dataset.
- For my transformation:-
- Change the dataset from varchar to integer, varchar to decimal.
- Change the raw.actor to make an average of the rating for each actor, and make it in group by and order by using the sql format.

```

1  > /* @bruin --
2  |
3  | name: actor
4  | type: duckdb.sql
5  |
6  | description: This asset loads a CSV file into a DuckDB database.
7  | columns:
8  |   - name: Actor
9  |     type: varchar
10  |     description: "name of actor"
11  |
12  |   - name: ActorID
13  |     type: varchar
14  |     description: "Source or connection through which contact was made"
15  |
16  |   - name: Rating
17  |     type: integer
18  |     description: "Date when contact was established"
19  |
20  |   - name: File
21  |     type: varchar
22  |     description: "Contact's job position or title"
23  |
24  |   - name: Year
25  |     type: integer
26  |     description: "Date when contact was established"
27  |
28  |   - name: Votes
29  |     type: integer
30  |     description: "Date when contact was established"
31  |
32  |   - name: FileID
33  |     type: varchar
34  |     description: "Date when contact was established"
35  |     name: pattern
36  |     value: "[A-Z][a-z]*"
37  |
38  | @bruin --

```

Fig 15. Bruin set up in the actor.sql file.

- To use the bruin tools as our pipeline database, for each of the transformation there is needed to introduce the column name before in the /* @bruin environment.
- The type also needed to be changes which is from duckdb.seed in the example.sql to duckdb.sql as it will be error in running the sql file.

2.1.3.1 The proof of transformations.

To differentiate the dataset before and after the transformation.

D from raw.actor;								
actor varchar	actor_id varchar	film varchar	year varchar	votes varchar	rating varchar	film_id varchar	_dlt_load_id varchar	_dlt_id varchar
Will Forte	nm0287182	She's Funny That Way	2014	24746	6.1	tt1767372	1738140648.183768	mt8ZpB+w4ErAEQ
Will Forte	nm0287182	Apartment Troubles	2014	505	4.3	tt2977110	1738140648.183768	vMpNDp6E4Ova0Q
Will Forte	nm0287182	Grown Ups 2	2013	142064	5.4	tt2191701	1738140648.183768	yw467iCmvsim1A
Will Forte	nm0287182	Nebraska	2013	113032	7.7	tt1821549	1738140648.183768	66NVanbjED3MZO
Will Forte	nm0287182	Cloudy with a Chance of Meatballs 2	2013	92007	6.4	tt1985966	1738140648.183768	oDRw2q5/HcmfQw
Will Forte	nm0287182	Life of Crime	2013	17336	5.8	tt1669207	1738140648.183768	5S4f4fHh0DJNTO
Will Forte	nm0287182	Run & Jump	2013	1223	6.3	tt2343158	1738140648.183768	/RUXRAMGOpnqA
Will Forte	nm0287182	The Watch	2012	121844	5.7	tt1298649	1738140648.183768	p1W8GZRCmGv1w
Will Forte	nm0287182	That's My Boy	2012	92383	5.5	tt1232200	1738140648.183768	wF5gBL19NvbPnw
Will Forte	nm0287182	Rock of Ages	2012	74320	5.9	tt1336608	1738140648.183768	Sjlgyxy83D6mhg
Will Forte	nm0287182	Tim and Eric's Billion Dollar Movie	2012	9534	5.4	tt1855401	1738140648.183768	E//7CGKzPuGFjw
Will Forte	nm0287182	A Good Old Fashioned Orgy	2011	21470	6.1	tt1231586	1738140648.183768	psPwPw7o3uAsPa
Will Forte	nm0287182	MacGruber	2010	42811	5.6	tt1470023	1738140648.183768	QNH2Zat4Wob0Q
Will Forte	nm0287182	Cloudy with a Chance of Meatballs	2009	207723	6.9	tt0844471	1738140648.183768	LycB5Kbm1sX8Q
Will Forte	nm0287182	Fanboys	2009	56341	6.6	tt0489049	1738140648.183768	1d77v1OKs0y1Uw
Will Forte	nm0287182	The Slammin' Salmon	2009	13578	6.4	tt1135525	1738140648.183768	J2DnUWrAsFqWg
Will Forte	nm0287182	Brief Interviews with Hideous Men	2009	3336	5.6	tt0790627	1738140648.183768	Cf+/rwydDarp0g
Will Forte	nm0287182	Baby Mama	2008	42608	6	tt0871426	1738140648.183768	10B3zJCAhZbW0
Will Forte	nm0287182	The Brothers Solomon	2007	10545	5.3	tt0784972	1738140648.183768	c/b3Vejpu1D9G5
Will Forte	nm0287182	Bearfest	2006	64409	6.3	tt0486551	1738140648.183768	/8JTUICDKaNLAA
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
Rosemary Forsyth	nm0287071	Texas Across the River	1966	1386	6.3	tt0011080	1738140648.183768	VPx4h4M6LCCFNg
Rosemary Forsyth	nm0287071	Shenandoah	1965	7984	7.3	tt0059711	1738140648.183768	AEQBPv7EzYfYA
Rosemary Forsyth	nm0287071	The War Lord	1965	3014	6.7	tt0059896	1738140648.183768	ggKC2+hwMDVDw
Will Forte	nm0287182	Scoob!	2020	20220	5.7	tt3152592	1738140648.183768	deGu63kQVQLRQ
Will Forte	nm0287182	The Willoughbys	2020	13833	6.4	tt5206260	1738140648.183768	/XbMb34cY+56HA
Will Forte	nm0287182	Booksmart	2019	96502	7.2	tt1489887	1738140648.183768	wqXR4fG3ejrBAQ
Will Forte	nm0287182	Good Boys	2019	61958	6.7	tt6977338	1738140648.183768	CnH1XLRe0+c7oA
Will Forte	nm0287182	The Lego Movie 2: The Second Part	2019	56264	6.6	tt3313498	1738140648.183768	19z5EP1L6wcbhw
Will Forte	nm0287182	The Laundromat	2019	45804	6.3	tt5865326	1738140648.183768	F3bkPjdIKtSzCQ
Will Forte	nm0287182	Extra Ordinary	2019	8033	6.4	tt8233874	1738140648.183768	xE3dUuZMZW0kTg
Will Forte	nm0287182	A Futile and Stupid Gesture	2018	14192	6.8	tt5566790	1738140648.183768	K9l1Ayz1BL4Pg
Will Forte	nm0287182	Luis and the Aliens	2018	2200	6	tt4410000	1738140648.183768	u/gwU23HCABz6g
Will Forte	nm0287182	Popstar: Never Stop Never Stopping	2016	55553	6.7	tt3960412	1738140648.183768	r9aM3Z1JyV48RW
Will Forte	nm0287182	Meanu	2016	45937	6.3	tt4139124	1738140648.183768	EoDKs3hnyvAAEA
Will Forte	nm0287182	My Life as a Zucchini	2016	22236	7.8	tt2321405	1738140648.183768	NzpbHhDQY4MTUA
Will Forte	nm0287182	The Ridiculous 6	2015	45009	4.8	tt2479478	1738140648.183768	/kaqk422pf0CCSA

Fig 16. The raw.actor dataset from the duckdb database. This is before the transformation happens which means the dataset is originally from raw .csv file obtained from kaggle.

D from actor_formatting;						
actor_name varchar	actor_id varchar	film varchar	year int32	votes int32	rating decimal(18,3)	movie_id varchar
50 Cent	nm1265067	Den of Thieves	2018	95618	7.000	tt1259528
50 Cent	nm1265067	Get Rich or Die Tryin'	2005	44370	5.400	tt0430308
50 Cent	nm1265067	Setup	2011	20549	4.400	tt1748197
50 Cent	nm1265067	Escape Plan 2: Hades	2018	28730	3.900	tt6513656
50 Cent	nm1265067	Popstar: Never Stop Never Stopping	2016	55553	6.700	tt3960412
50 Cent	nm1265067	Spy	2015	231500	7.000	tt3079380
50 Cent	nm1265067	Southpaw	2015	220037	7.400	tt1798684
50 Cent	nm1265067	The Prince	2014	17650	4.600	tt1085492
50 Cent	nm1265067	Escape Plan	2013	228881	6.700	tt1211956
50 Cent	nm1265067	Last Vegas	2013	128857	6.600	tt1204975
50 Cent	nm1265067	The Frozen Ground	2013	59071	6.400	tt2005374
50 Cent	nm1265067	Fire with Fire	2012	25340	5.600	tt1925431
50 Cent	nm1265067	All Things Fall Apart	2011	2497	5.500	tt1606390
50 Cent	nm1265067	Morning Glory	2010	75059	6.500	tt1126618
50 Cent	nm1265067	13	2010	40550	6.100	tt0798817
50 Cent	nm1265067	Twelve	2010	11857	5.700	tt1407084
50 Cent	nm1265067	Gun	2010	3604	3.800	tt1560954
50 Cent	nm1265067	Caught in the Crossfire	2010	1155	4.400	tt1449379
50 Cent	nm1265067	Dead Man Running	2009	3992	6.000	tt1311699
50 Cent	nm1265067	Righteous Kill	2008	85179	6.000	tt1034331
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.
Zulay Henao	nm1748489	Grand Isle	2019	2772	4.700	tt8380776
Zulay Henao	nm1748489	Grizzly Park	2008	2270	4.000	tt0915460
Zulay Henao	nm1748489	Feel the Noise	2007	3549	2.900	tt0756703
Zulay Henao	nm1748489	S. Darko	2009	13307	3.600	tt1231277
Zulay Henao	nm1748489	Fighting	2009	32023	5.600	tt1082601
Zulay Henao	nm1748489	Boy Wonder	2010	8904	6.800	tt1456060
Zulay Henao	nm1748489	Takers	2010	58457	6.200	tt1135084
Zulay Henao	nm1748489	The Single Moms Club	2014	5899	5.400	tt2465140
Zulay Henao	nm1748489	Destined	2016	118	4.800	tt2515456
Zulay Henao	nm1748489	True Memoirs of an International Assassin	2016	20749	5.900	tt1542768
Zulay Henao	nm1748489	Grow House	2017	323	4.700	tt4274656
Zulay Henao	nm1748489	Beyond White Space	2018	1535	4.500	tt1977094
Zuleikha Robinson	nm0733196	The Merchant of Venice	2004	35228	7.000	tt0379889
Zuleikha Robinson	nm0733196	Timecode	2000	6489	6.000	tt0220100
Zuleikha Robinson	nm0733196	Slash	2002	1512	4.200	tt0301848
Zuleikha Robinson	nm0733196	Hidalgo	2004	77730	6.700	tt0317648

Fig 17. The raw.actor after transformation mentioned from the table above, which is changing the column format for year and rating. Next, the transformation is the column name.

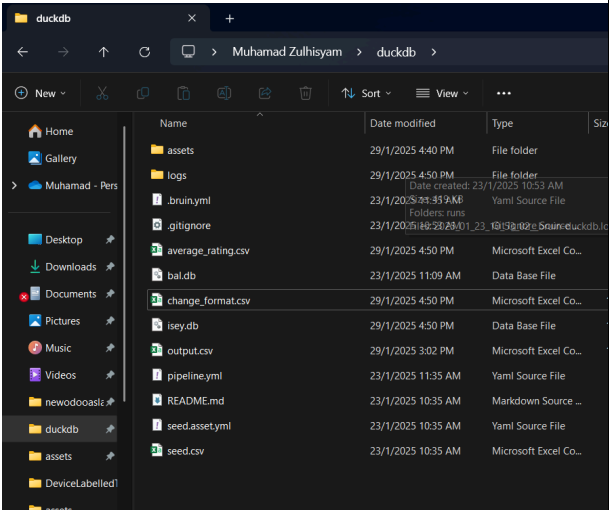
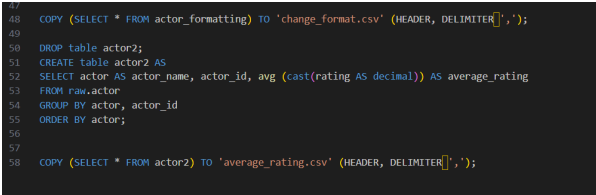
D from actor2;

actor_name varchar	actor_id varchar	average_rating double
50 Cent	nm1265067	5.579166666666667
A Martinez	nm0553436	5.71
A.J. Buckley	nm0118568	5.480769230769231
A.J. Cook	nm0176882	5.569230769230769
A.J. Langer	nm0486277	5.15
AJ Michalka	nm1404488	6.044444444444444
Aadila Dosani	nm4602010	6.1
Aaliyah	nm0004691	5.7
Aamir Khan	nm0451148	6.810416666666667
Aaron Ashmore	nm0039148	5.82
Aaron Douglas	nm0234928	5.8
Aaron Eckhart	nm0001173	6.315789473684211
Aaron Jeffery	nm0420279	5.95
Aaron Lohr	nm0517909	6.111111111111111
Aaron Moorhead	nm1823264	5.34
Aaron Paul	nm0666739	6.09
Aaron Pedersen	nm0669853	5.677777777777778
Aaron Ruell	nm1088052	5.066666666666666
Aaron Schwartz	nm0777125	5.628571428571429
Aaron Sorkin	nm0815070	7.4
.	.	.
.	.	.
.	.	.
Zoe Levin	nm4534201	6.675
Zoe Lister-Jones	nm1830380	5.916666666666667
Zoe Margaret Colletti	nm2292661	6.3
Zoe McLellan	nm0572721	5.644444444444445
Zoe Perry	nm1409365	5.9
Zoe Saldana	nm0757855	6.319047619047619
Zoe Tapper	nm1483730	5.77
Zoe Telford	nm0854364	6.125
Zoe Wanamaker	nm0910738	6.4875
Zoey Deutch	nm3614913	6.3
Zoey Luna	nm10656367	4.4
Zofia Wichlacz	nm5851109	6.2
Zohar Shtrauss	nm1364521	6.646153846153846
Zoie Palmer	nm1235776	5.5
Zola Williams	nm6880265	7.333333333333333
Zoey Deschanel	nm0221046	6.38
Zosia Mamet	nm0541082	5.9375
Zsa Zsa Gabor	nm0001248	5.846153846153846
Zulay Henao	nm1748489	4.957142857142857
Zuleikha Robinson	nm0733196	6.1
9623 rows (40 shown)		3 columns

Fig 18. The second transformation of the dataset which is the average of all the actors in the dataset. As depicted in Fig 18. There are 9623 columns after the transformations, but previously the row was at 191682 rows (40 shown).

2.1.4 Data Serving

In the context of data engineering, serving data is making data available to users. Data analysts, data engineers (models and machine learning), and reverse ETL are the three primary user categories. Data analysts will create dashboards, reports, and statistical analyses [3].

Process	Explanation
 Fig 19. The folder of duckdb after the serve.	- As for the data serving, I implement the sql to save the table from the database in the duckdb folder in .csv. - As you can see at the left, the name of the newly .csv file is average_rating.csv and the change_formatting.csv.
 Fig 20. The actor.sql file	- The sql code showed how to serve the dataset as a .csv file. - It is using the same actor.sql file from above. - The drop table was to make sure there is no repeated table created in the folder that can cause an error.

```
COPY (SELECT * FROM actor_formatting) TO 'change_format.csv' (HE/
```

Fig 21. The sql code to save the edited dataset in .csv

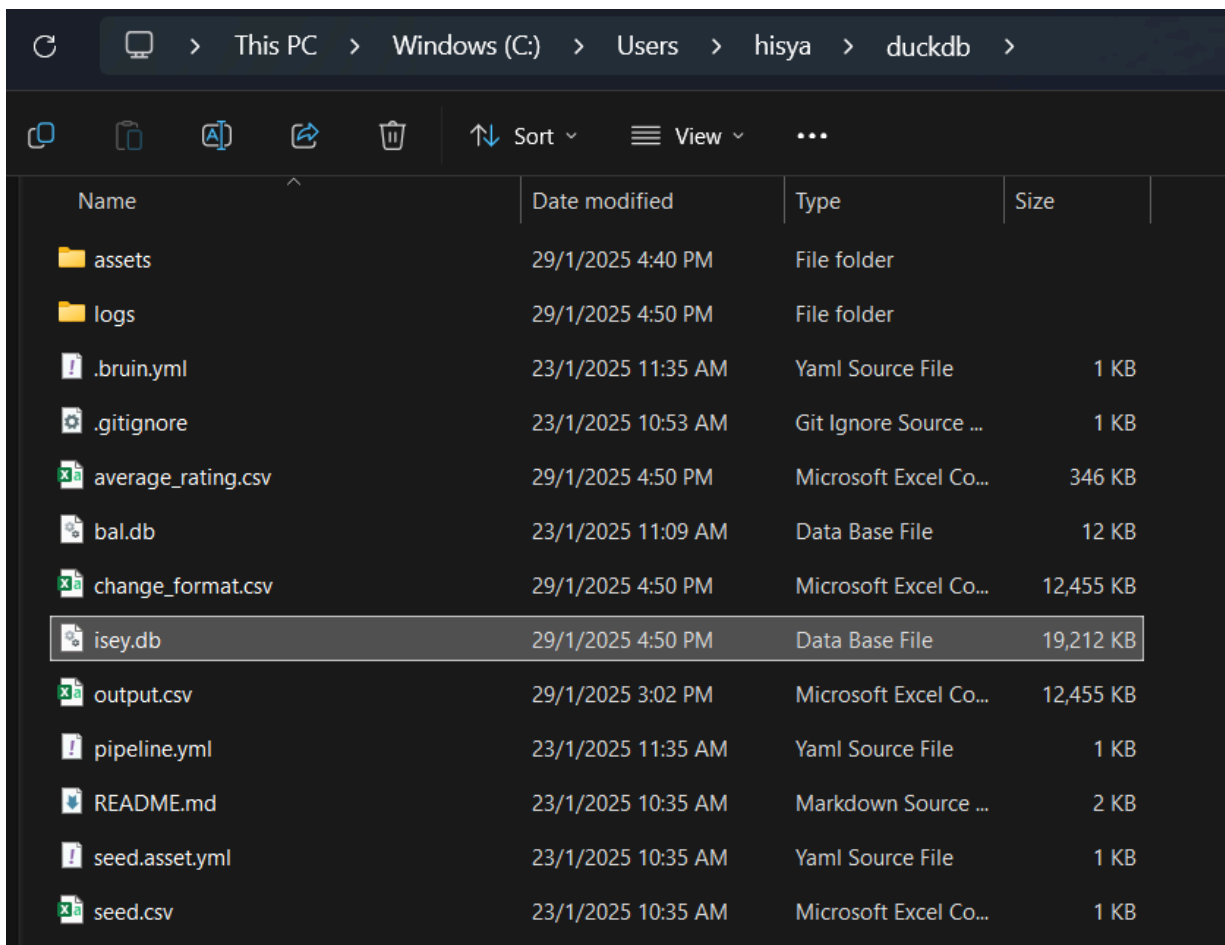
- For the proof of the execution and the ingestion of successfully is at Appendices below.

Appendices

All the files and configurations can be found in the link below:-

Big Data Appendices

- The duckdb folder showed all the files created until served data.
- The assets folder in the duckdb showed the file that needed to be created in order to make the ingestion of new dataset and transformation possible.



The screenshot shows a Windows File Explorer window with the address bar set to 'This PC > Windows (C:) > Users > hisya > duckdb'. The file list includes folders 'assets' and 'logs', and various files. The file 'isey.db' is selected and highlighted.

Name	Date modified	Type	Size
assets	29/1/2025 4:40 PM	File folder	
logs	29/1/2025 4:50 PM	File folder	
.bruin.yml	23/1/2025 11:35 AM	Yaml Source File	1 KB
.gitignore	23/1/2025 10:53 AM	Git Ignore Source ...	1 KB
average_rating.csv	29/1/2025 4:50 PM	Microsoft Excel Co...	346 KB
bal.db	23/1/2025 11:09 AM	Data Base File	12 KB
change_format.csv	29/1/2025 4:50 PM	Microsoft Excel Co...	12,455 KB
isey.db	29/1/2025 4:50 PM	Data Base File	19,212 KB
output.csv	29/1/2025 3:02 PM	Microsoft Excel Co...	12,455 KB
pipeline.yml	23/1/2025 11:35 AM	Yaml Source File	1 KB
README.md	23/1/2025 10:35 AM	Markdown Source ...	2 KB
seed.asset.yml	23/1/2025 10:35 AM	Yaml Source File	1 KB
seed.csv	23/1/2025 10:35 AM	Microsoft Excel Co...	1 KB

Fig 22. Proof of the database file isey.db exists. This isey.db cannot be uploaded in the GDrive as it is not readable in the usual file because it has to be accessed through duckdb cmd environment as shown in the [Fig 16. Fig 17. And Fig 18.](#)

The trailer video on what is happening:-

 BigData_Assignment_G7.mp4

References

[1] DATANOSCO, "Bruin: The unified Analytic Platform, for Ingestion, Transformation, Data Quality and Governance," *YouTube*, Jan. 23, 2025.

<https://www.youtube.com/watch?v=JZXcAoMm9-4>

[2] TIBCO, "What is Data Transformation? | TIBCO," *TIBCO*, 2024.

<https://www.tibco.com/glossary/what-is-data-transformation>

[3] "ContentKeeper," *Begincodingnow.com*, 2025. <https://begincodingnow.com/data-serving/> (accessed Jan. 27, 2025).